

Cierre de compra desde la app

Todo lo que falta para que una clienta compre desde la app, en orden de ejecución. Cada tarea dice qué es, para qué sirve y cómo hacerla. Complementa el `COMPRA_EN_APP_SPEC.md`, que sigue siendo la fuente de verdad.

Estado al 29/08/2026

El backend está terminado y probado: **527 tests pasando**. Están hechos el callback de OAuth, los tres webhooks de privacidad, la emisión y limpieza de cupones, el carrito, el checkout en WebView y la atribución de la venta.

Lo que falta **casi no es código**. Son cuatro trámites en Railway y en el panel de Tienda Nube, dos preguntas al centro, y un ensayo completo en la tienda de prueba. Nada de lo construido tocó Tienda Nube todavía.

TRÁMITE lo hacés vos, fuera del código

CÓDIGO lo hago yo

DECISIÓN la contesta el centro

Bloque 1 · Destruir lo que ya está construido

Minutos de trabajo cada uno, y todo lo demás espera atrás. El orden interno importa: el secret va antes que los webhooks.

1. Cargar las credenciales de Tienda Nube en Railway

TRÁMITE

Qué es. Dos variables de entorno con las claves de la app de partners: `TIENDANUBE_CLIENT_ID` y `TIENDANUBE_CLIENT_SECRET`. Identifican a la app, no a una tienda.

Para qué. Sin el secret, el callback no puede canjear el código de instalación por el token, y los tres webhooks contestan 401 a todo. Hoy no están cargadas en ningún lado: el backend de producción tiene cero variables `TIENDANUBE`.

Cómo

- En **partners.tiendanube.com**, entrá a la app «Ame-App-Clientes» (app_id 39429) y abrí la sección **Claves de Acceso**.
- Copiá el *client id* y el *client secret*.
- En Railway, entrá al servicio **Plataforma-Estetica** (el backend) y agregalas como variables de entorno. El deploy se rehace solo.
- Verificá que quedaron con este comando en el servicio:

```
python manage.py shell -c "from django.conf import settings;
print(bool(settings.TIENDANUBE_CLIENT_SECRET))"
```

Va primero que todo lo demás. Si registrás los webhooks antes que el secret, Tienda Nube recibe 401 y reintenta cada pedido dieciséis veces durante 48 horas.

2. Apuntar la URL de redirección al callback

TRÁMITE

Qué es. La dirección adonde Tienda Nube manda al comerciante después de que autoriza la app. Hoy apunta a la página del panel de partners.

Para qué. Es lo que convierte la instalación en automática. Sin esto, vincular una tienda exige copiar a mano un código que **vence a los cinco minutos** y pegarlo en un comando.

Cómo

- En el panel de partners, editá la app y buscá el campo de **URL de redirección** (también aparece como *redirect URI*).

- Pegá exactamente esta dirección:

```
https://plataforma-estetica-production.up.railway.app/api/integraciones/tiendanube/oauth/callback/
```

- Guardá. No hace falta nada del lado nuestro: la ruta ya existe y responde.

Podés comprobar que responde abriéndola en el navegador: tiene que devolver una página que dice «Falta el código de instalación». Eso significa que está viva y esperando.

3. Registrar las tres URLs de webhook

TRÁMITE

Qué es. Los tres avisos obligatorios de privacidad que Tienda Nube exige a toda app: cuando desinstalan la app, cuando piden borrar los datos de un comprador, y cuando piden un informe de esos datos.

Para qué. Son requisito para homologar la app. Además, el primero es el que apaga la integración sola cuando el centro desinstala, para que no queden credenciales vivas.

Cómo

- En el panel de partners, cargá una URL por evento:

```
.../api/integraciones/tiendanube/webhooks/store-redact/  
.../api/integraciones/tiendanube/webhooks/customers-redact/  
.../api/integraciones/tiendanube/webhooks/customers-data-request/
```

- Las tres cuelgan del mismo dominio de Railway que el callback.

Después del secret, nunca antes. Los tres validan la firma HMAC del pedido, y sin el secret cargado rechazan todo — incluidos los pedidos legítimos de Tienda Nube.

4. Crear el servicio de cron que limpia los cupones

TRÁMITE

Qué es. Un servicio de Railway que corre una vez por hora el comando `limpiar_cupones_app`.

Para qué. Cada vez que una clienta toca «Comprar» y no termina la compra, queda un cupón vivo en el panel del centro. Sin limpieza, en unos meses hay miles. El comando ya existe y está probado; falta que algo lo corra.

Cómo

- En Railway, creá un servicio nuevo apuntado al mismo repo.
- Configuralo con este archivo, que ya está en el repo:

```
backend/railway.cupones.cron.json
```

- Programalo cada una hora. Los cupones duran una hora, así que ninguno sobrevive más de dos.
- Es el mismo patrón que ya usa el servicio **conto-sync**.

La fila local del cupón no se borra nunca: es la que después responde cuántas compras se empezaron y no se terminaron. Solo se borra el cupón del lado de Tienda Nube.

Bloque 2 · Ordenar antes de tocar Tienda Nube

Una sola tarea, y es el mejor momento para hacerla: producción todavía no tiene ninguna tienda vinculada.

5. Darle a la tienda de prueba su propio centro

DECISIÓN

Qué es. Crear en producción un centro estético aparte —por ejemplo «Centro de pruebas»— al que se vincule la tienda demo, en vez de colgarla del centro de AME.

Para qué. Es una barrera, no prolijidad. El emparejador de productos y el callback trabajan siempre **dentro del centro de la integración**. Con la demo en su propio centro, ningún ensayo puede marcar productos reales de AME como comprables contra una tienda de prueba, aunque te equivoques de comando.

Cómo

- Entrá al admin de Django de producción, a **Centros estéticos**, y creá uno nuevo para las pruebas.
- Creale una sucursal, que es lo que necesita la integración.
- Cuando instales la app en la tienda demo (tarea 8), arrancá la instalación desde ese centro.

Si lo hacés al revés —vincular AME primero y la demo después contra el mismo centro— la integración de AME se pisa en silencio y las clientas terminarían comprando en la tienda de prueba. Con dos centros, el sistema lo rechaza solo y te dice de quién es la tienda.

Bloque 3 · Preparar la tienda de prueba

La tienda demo hoy no puede cobrar ni tiene catálogo. Sin esto no se puede ensayar una compra.

6. Instalar un medio de pago en la tienda demo

TRÁMITE

Qué es. Una forma de cobrar en la tienda de prueba: transferencia bancaria o efectivo alcanzan.

Para qué. Hoy la demo **no tiene ninguno**, y por eso el checkout llega al paso «Pago» y se queda ahí. Es el bloqueo que impide probar todo lo que viene después del pago: la vuelta a la app, la pantalla de confirmación y la atribución.

Cómo

- Entrá al panel de la tienda demo (amedemo.mitiendanube.com).
- Andá a la tienda de aplicaciones y buscá un medio de pago manual (transferencia o efectivo).
- Instalalo y activalo.

Tienda Nube sacó los medios de pago manuales de su lista propia y ahora son aplicaciones de su tienda, así que hay que instalarlos como cualquier otra app.

7. Cargar tres o cuatro productos en la tienda demo

TRÁMITE

Qué es. Productos de prueba en la tienda demo, con **el mismo nombre** que tienen en la plataforma.

Para qué. La demo tiene un solo producto, un «Serum» de \$100. Con tres o cuatro se puede armar un carrito de varias líneas, que es el caso que nunca se probó: la app agrega los productos de a uno, en secuencia.

Cómo

- En el panel de la tienda demo, creá los productos a mano.
- Usá exactamente el mismo nombre que tienen en el inventario de la plataforma, así el emparejador los reconoce solo.
- Después corré el emparejador, que primero propone y no escribe nada:

```
python manage.py emparejar_variantes_tiendanube --centro
```

- Revisá la lista y, si está bien, aplicá:

```
python manage.py emparejar_variantes_tiendanube --centro --aplicar
```

Por API no se puede. El token de la app solo tiene permiso de lectura de productos y escritura de cupones, así que los productos se cargan a mano en el panel.

Bloque 4 · El ensayo completo, en la tienda de prueba

Todo lo que se pueda romper, que se rompa acá. Ninguna de estas tareas toca la tienda ni el catálogo de AME.

8. Desinstalar y volver a instalar la app en la demo

TRÁMITE

Qué es. El ciclo completo de instalación, hecho contra la tienda de prueba.

Para qué. Es la única forma de probar de verdad lo que se construyó: desinstalar hace que Tienda Nube dispare el webhook de borrado, y volver a instalar manda un código de instalación real al callback. Hasta ahora las dos cosas solo se probaron con tests.

Cómo

- En el admin de Django de producción, entrá a **Instalaciones de Tienda Nube iniciadas** y creá una para el centro de pruebas. Al guardar te manda directo a Tienda Nube.
- Autorizá la app con una cuenta que administre la tienda demo.
- Tenés **quince minutos** desde que guardás hasta que autorizás.
- Después probá la desinstalación desde el panel de la tienda.

Qué mirar: que al desinstalar la integración quede apagada y sin token, que los cupones ya emitidos **sobrevivan**, y que al reinstalar la tienda se vincule sola, sin correr ningún comando.

9. Hacer una compra de prueba, primero en \$0

TRÁMITE

Qué es. Una compra completa desde la app: carrito, cupón, checkout y pago.

Para qué. Es lo que revela **la dirección real de la página de gracias** de Tienda Nube. Hoy la app la tiene adivinada, y si está mal, la clienta termina viendo la página de Tienda Nube en vez de la nuestra — justo la sensación que todo el WebView existe para evitar.

Cómo

- Poné el descuento del segmento en 100% para que la compra dé \$0 y no gastes plata.
- Comprá desde la app, con el teléfono, hasta el final.
- Anotá la dirección de la página de gracias y pasámela: corrijo la app con el valor real.
- Repetí con el medio de pago puesto y un monto chico: en \$0 el checkout puede saltarse el paso de pago y comportarse distinto.

La compra la tenés que hacer vos: hay que cargar un mail y una dirección de entrega, y eso no lo completo yo.

10. Ensayar la vuelta de la venta con el simulador

TRÁMITE

Qué es. Un comando que arma el comprobante que mandaría Conto por una venta hecha con un cupón, y lo importa por el camino de siempre.

Para qué. Una compra en la tienda demo **nunca vuelve por Conto**: Conto mira la cuenta de AME, que está atada a la tienda real. Sin el simulador, la mitad de atrás —el ingreso, la atribución, la clienta— queda sin probar hasta la primera venta de producción.

Cómo

- Buscá en el admin, en «Cupones de la app», el código de un cupón real.
- Corré el simulador con ese código:

```
python manage.py simular_venta_app --cupon APP-XXXXXXX
```

- Vas a ver la transacción creada, la venta marcada como originada en la app, el cupón marcado como usado y la clienta identificada.
- Cuando termines de mirar, **deshacelo siempre**:

```
python manage.py simular_venta_app --cupon APP-XXXXXXX --deshacer
```

Genera ingresos reales en el módulo financiero. Por eso se niega a correr en producción salvo que se lo fuerce, y por eso hay que deshacerlo: cada ensayo sin limpiar es plata que no entró ensuciando los números.

Bloque 5 · Verificar antes de medir

No depende de ningún trámite ni de ninguna venta. Conviene hacerlo cuanto antes.

11. Confirmar que Conto manda el código del cupón

TRÁMITE

Qué es. Un comando de solo lectura que le pregunta a Conto por un comprobante real y verifica que traiga los cuatro campos nuevos, entre ellos cupon.

Para qué. Toda la atribución de ventas de la app descansa en ese campo. El equipo de Conto lo agregó en agosto y avisó, con buen criterio, que **no pudo compilar**. Si nunca llegó, la medición va a mostrar ceros para siempre y nadie se va a enterar.

Cómo

- Corrélo contra producción, que es donde vive Conto:
`python manage.py verificar_conto`
- Buscá en la salida las líneas `ventas.cupon`, `ventas.descuento_cupon`, `ventas.origen_venta` y `ventas.app_origen`.
- Si alguna dice «falta», avisale a Conto antes de seguir.

No escribe nada, solo hace consultas. En la base de desarrollo no corre porque el token de Conto se deprecó a propósito: Conto vive solo en producción.

Bloque 6 · Lo que hago yo

Código puro. No depende de ningún trámite, salvo donde se aclara.

12. Mejorar el emparejador de productos

CÓDIGO

Qué es. Agregarle dos señales al comando que vincula cada producto de la plataforma con el de Tienda Nube: el **SKU** y el **precio**.

Para qué. Hoy empareja solo por parecido de nombre. El SKU ya se lee de Tienda Nube y **no se usa**: un SKU que coincide no es un parecido, es una certeza. Y el precio no se mira en ningún momento, así que un producto de \$19.000 puede emparejarse con uno de \$100 si los nombres se parecen.

Cómo

- SKU exacto pasa a ser match directo, sin pasar por similitud de nombre.
- Sin SKU, se empareja por nombre como ahora.
- El precio queda de testigo: una diferencia grande baja el match a «dudoso» aunque el nombre coincida al 100%.
- Son unas dos horas.

Conviene antes de emparejar el catálogo real de AME (tarea 14), porque ese emparejamiento se hace una vez y después queda.

13. La medición de cuánto vendió la app

CÓDIGO

Qué es. El módulo que responde cuánto vendió la app, qué productos, a qué clientes y con cuánto descuento.

Para qué. Es la métrica que justifica el proyecto entero y la que decide si algún día conviene construir un checkout propio. Sin esto, en tres meses no hay forma de separar lo que vendió la app de lo que vendió la web.

Cómo

- Se valida con el simulador y con las compras de la demo, así que llega a producción probada en vez de estrenarse con la primera venta real.
- Requiere que la tarea 11 esté confirmada primero.

La pregunta que conviene contestar no es «cuánto vendió la app» sino el **ticket promedio con cupón contra sin cupón**: distingue las compras que el descuento trajo de las que iban a pasar igual y solo le costaron margen al centro.

Bloque 7 · La tienda real de AME

Último, y a propósito: para cuando se llega acá, todo lo demás ya se ensayó en la tienda de prueba.

14. Instalar la app y emparejar el catálogo real

TRÁMITE

Qué es. La misma instalación de la tarea 8, pero contra la tienda de AME, y después el emparejamiento de los 31 productos activos.

Para qué. Es lo que enciende la compra de verdad. Acá el catálogo no da trabajo: la tienda real **ya tiene** los productos cargados, así que el emparejador los cruza contra el inventario de la plataforma.

Cómo

- Arrancá la instalación desde el admin, con el centro de AME.
- Corré el emparejador en modo propuesta y revisá la lista:
`python manage.py emparejar_variantes_tiendanube --centro`
- Mirá especialmente los que tengan porcentaje bajo, y los «dudosos».
- Aplicá cuando estés conforme, y completá a mano desde el CRM los que quedaron sin emparejar.

Bloqueado por una pregunta sin responder: si Tienda Nube permite instalar una app «en desarrollo» en una tienda real. Si la respuesta es que no, hay que homologar la app primero.

15. Una compra real con un producto barato

TRÁMITE

Qué es. Una compra de verdad en la tienda de AME, hecha desde la app, y después cancelada.

Para qué. Es lo único que confirma el último eslabón: que Conto manda el código del cupón en el comprobante y que la venta se atribuye a la app y a la cliente. De 232 comprobantes ya importados, ninguno trae cupón, así que no hay dato histórico que lo pruebe.

Cómo

- Avisale al centro antes, para que no despachen una orden de prueba.
- Comprá el producto más barato desde la app.
- Cancelá la orden desde el panel de Tienda Nube: eso devuelve el stock.
- Revisá que la venta haya vuelto por el sync y quedado atribuida.

Es el paso que cierra el proyecto. Después de esto, la app vende.

En paralelo · Dos preguntas para el centro

No bloquean el trabajo técnico, pero una de las dos define si la app descuenta algo.

16. ¿El descuento de la app se suma al de transferencia?

DECISIÓN

Qué es. El centro ya da 10% por pagar con transferencia. Hay que decidir si el descuento de la app se suma a ese o lo reemplaza.

Para qué. Sobre un producto de \$20.900 con 15% de app, sumarlos es un 25% acumulado. Puede ser exactamente lo que el centro quiere —empuja las dos cosas que le convienen— o margen que se va sin que nadie lo haya decidido.

Cómo

- Preguntarle al centro.
- La respuesta se carga en la integración, en el campo que decide si los cupones se combinan con otras promociones.
- Después hay que definir el porcentaje del segmento general.

Mientras no se responda, el descuento queda en 0% y la app no descuenta nada. O sea que el incentivo para instalarla, que es el motivo del proyecto, hoy no existe.

17. ¿Las ofertas del CRM existen también en Tienda Nube?

DECISIÓN

Qué es. Si el centro carga una oferta de producto en el CRM, hay que saber si también la carga en su tienda de Tienda Nube.

Para qué. Si no, la app muestra un precio con descuento que el checkout no va a respetar, porque el cupón se aplica sobre el precio de Tienda Nube. La clienta vería cambiar el precio justo cuando va a pagar.

Cómo

- Preguntarle al centro.
- Si la respuesta es que no, es un cambio de una línea: la app pasa a mostrar el precio de lista.

Hoy no muerde porque ningún producto tiene oferta activa. Muerde el día que alguien prenda una.

El orden importa poco dentro de cada bloque, y mucho entre bloques. El bloque 1 destraba todo lo demás y son minutos. El bloque 2 es la única tarea que conviene hacer *antes* de tocar Tienda Nube, porque después cuesta más. Y la tienda real queda al final a propósito: para cuando se llega ahí, cada paso ya se ejecutó al menos una vez contra la tienda de prueba.